

EXECUTIVE BRIEF

Project Livewire

Real-Time Multimodal AI Chat
Powered by the Gemini Live API

Heiko Hotz

AI Engineer, Google Cloud | Faculty, London Business School | O'Reilly Author
heikohotz.com

What Livewire Is

I created Project Livewire to answer a question that most Gemini demos don't address: what does it really take to deploy a streaming multimodal AI app in a production-ready setup?

Livewire is a real-time chat application where you speak naturally, share your webcam or screen, and receive instant spoken responses. You can interrupt the AI mid-sentence, just like a real conversation. The system combines voice, video, text, and live tool calls over a single persistent connection. It runs on Google Cloud Run with proper secret management, session lifecycle handling, and containerised deployment.

You can see it as a reference example of what the Gemini Live API can do when you go beyond simple request and response demos and tackle the real engineering challenges of streaming, concurrent, multimodal interaction.

Repository	github.com/heiko-hotz/project-livewire
AI Model	Gemini Live API (multimodal, streaming)
Stack	Vanilla JS (client) + Python WebSocket (server)
Deployment	Google Cloud Run via Cloud Build
Licence	Apache 2.0

Why I Built It

The Gemini Live API creates a persistent, two-way WebSocket connection that streams audio, video, and text at the same time. The model gives audio responses in real time. This is a big shift from turn-based conversational AI and brings up engineering questions that documentation alone can't answer.

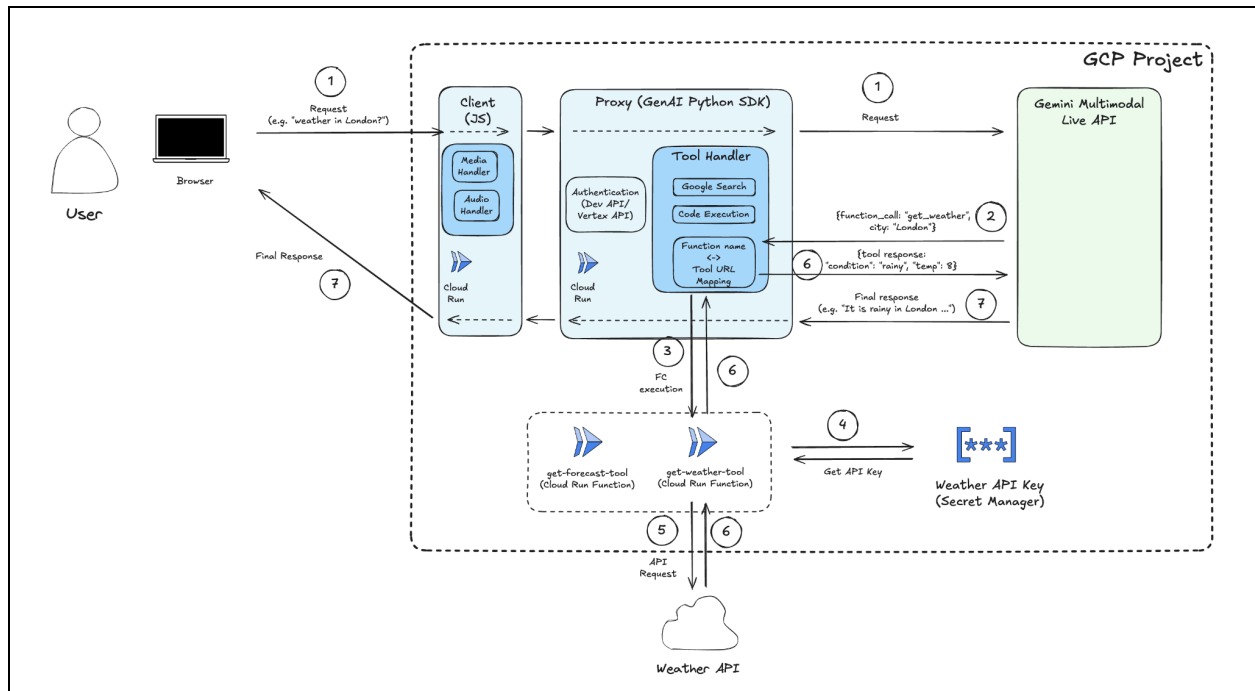
I wanted to understand those questions firsthand. How do you handle audio buffer management when the user interrupts mid-stream? How do you execute external tool calls without blocking the audio pipeline? How do you structure the session lifecycle so that timeouts, disconnections, and reconnections all resolve cleanly? Livewire is the result of working through each of these problems in a realistic deployment.

The Live API's interruptibility is what makes it feel fundamentally different from anything that came before it. Users talk over the model mid-response, and the system has to stop playback, clear the buffer queue, and resume cleanly — all within milliseconds. Getting this right was one of the hardest and most rewarding parts of the build.

Architecture

I designed Livewire as a three-tier system where each part has a clear, single responsibility. The security model is built into the architecture from the start, not added later.

Tier	Technology	Responsibility
Client	Vanilla JS + HTML/CSS	UI rendering, media capture (mic, webcam, screen), WebSocket connection, audio playback via WebAudio API
Server	Python + websockets	WebSocket proxy to Gemini, session management, tool routing, credential access via Secret Manager
Tools	Cloud Functions	Modular external capabilities (weather, calendar) invoked on Gemini's function-call requests



The client never stores API keys. Every Gemini interaction goes through the server, which gets credentials from Google Cloud Secret Manager at runtime. This keeps sensitive data out of client code and container images. I recommend this approach for any production deployment.

Design principle: I implemented tools as standalone Cloud Functions rather than inline server logic. This gives you independent deployment, natural per-tool scaling, and clear security boundaries. The tool handler routes function calls dynamically from configuration — adding a new tool requires zero code changes in the handler itself.

Key Capabilities

Multimodal Streaming Input

Users can use voice, text, webcam video, and screen sharing all in one session. I capture audio with the WebAudio API and an AudioWorklet processor for efficient, real-time chunks. Video frames from the webcam or screen share go along with the audio stream over the same

WebSocket connection. Gemini processes everything at once, so it can refer to what you show on screen while replying to what you say.

Real-Time Audio Output

The Live API streams audio responses as PCM16 chunks. I built the client's audio streamer to manage a buffer queue and play back chunks in order using WebAudio, which gives smooth, low-latency spoken output. The system also handles edge cases like buffer underruns, context switches, and stopping playback right away if the user interrupts.

Interruption Handling

If you start speaking while the model is still responding, the system detects the interruption using Gemini's interrupted signal. The server sends this to the client, which stops audio playback, clears the buffer, and shows visual feedback right away. The conversation then continues smoothly from your new input. I use an asynchronous tool-processing queue to keep the response pipeline responsive, even when tools are running at the same time.

Integrated Tool Calling

Gemini can call external tools during a conversation by making function calls. The server's tool handler sends these to the right Cloud Function endpoint. Tool responses go back into the Gemini session, so the model can include live data like weather or calendar events in its spoken reply. I use an async queue for tool calls, so the system keeps streaming audio while a tool runs in the background.

Deployment and Infrastructure

I packaged both the client and server with Docker and deployed them to Google Cloud Run using Cloud Build. The whole pipeline is set up in cloudbuild.yaml files, so deployments are repeatable and only need one command.

Client container: An nginx:alpine image serves the static files. The mobile-optimized UI is the default entry point, showing the voice-first design approach.

Server container: Python 3.11-slim runs the WebSocket server on port 8081. Only runtime dependencies are included, so the image stays small.

Secret management: Google Cloud Secret Manager stores all API keys. The server gets them at runtime using a dedicated service account with only the Secret Manager Secret Accessor role.

Configuration priority: Secret Manager, then environment variables, then .env file. The same codebase works the same way in local development, CI/CD, and production, with no need for code-level branching.

Production pattern: The configuration loading priority means I never maintain separate configs for different environments. The code simply resolves credentials from the most authoritative source available at runtime. This eliminates an entire class of deployment bugs.

Technology Choices

Vanilla JavaScript for the client: The app's complexity is in media handling and WebSocket communication, not UI state management. I chose Vanilla JS on purpose because it removes build tools, keeps the bundle small, and makes the Gemini Live API integration easy to read for anyone checking out the code. For a reference project, clarity is more important than framework features.

Python WebSocket proxy for the server: The server's main job is to keep API credentials off the client and handle tool execution. Python's async tools (like asyncio and the websockets library) and strong support in the google-genai SDK made it the obvious

choice. The proxy setup also makes it easy to add server-side logging, rate limiting, and session management.

Cloud Functions for tools: By making each tool its own Cloud Function, I get independent deployment, versioning, easy scaling for each tool, and clear security boundaries. The downside is a bit more cold-start delay for rarely used tools, but in a voice-first app where responses stream in, the audio buffer hides the delay from users.

What This Demonstrates

Livewire is a reference project for engineers and architects who want to see how to build production-ready apps on streaming multimodal AI APIs. It tackles engineering challenges that basic documentation and simple demos don't address.

Challenge	How Livewire Solves It
Real-time audio pipeline	AudioWorklet-based capture, buffer-queued playback, graceful interrupt handling
Concurrent multimodal input	Simultaneous audio, video, and text over a single WebSocket with server-side session state
Secure credential management	Proxy architecture with Secret Manager; the client never touches API keys
Tool execution during streaming	Async tool queue processes function calls without blocking audio responses
Session lifecycle	Clean creation, error recovery, timeout handling, and teardown of Gemini sessions
Cloud-native deployment	Dockerised client and server, declarative Cloud Build configs, environment-agnostic configuration

Go Deeper

GitHub repository: github.com/heiko-hotz/project-livewire. You'll find the full source code, deployment guides, and documentation for both local and cloud setups.

Gemini Multimodal Live Dev Guide:
github.com/heiko-hotz/gemini-multimodal-live-dev-guide. This is a companion project I made to explain the core ideas behind the Live API. Livewire builds on these concepts with a production-ready setup.

Gemini Live API documentation: ai.google.dev/gemini-api/docs/multimodal-live. This is Google's official reference for the streaming multimodal API that powers Livewire.

About Heiko

I'm an AI Engineer at Google Cloud, where I help enterprise customers design and deploy production AI systems on Vertex AI. I also teach generative AI at London Business School and write about production AI patterns for O'Reilly Media. With 24 years in tech and over 9 years in AI/ML, I focus on bridging the gap between what AI can do in a demo and what it takes to make it work reliably at scale.

You can see more of my work at heikohotz.com or connect with me on LinkedIn.

Interested in building with streaming multimodal AI?

Feel free to reach out at heikohotz.com or check out the full source code on GitHub.